

CXLoom Design Understanding

This note consolidates the current understanding of the two source documents:

- CXL Pod一致性 - 论文设计.pdf -> LoomMem
- LeoPar_to_EuroSys.pdf -> LeoPar, to be adapted and renamed as LoomPar

Its purpose is to give us an implementation-oriented shared model before coding the full CXLoom system.

1. CXLoom: Unified System View

CXLoom consists of two peer subsystems:

- LoomMem : manages how shared data is represented, allocated, cached, synchronized, and made visible across hosts.
- LoomPar : manages how computation over that shared data is created, placed, synchronized, and completed across hosts.

The intended symmetry is:

- Mem : how shared data is managed
- Par : how computation over shared data is organized

Architecturally, the system is:

- a software-coherent CXL shared-memory substrate
- plus a distributed thread runtime built directly on top of it

2. LoomMem: What It Must Provide

From the LoomMem design document, the V1 system model is:

- One physical multi-NUMA server is partitioned into multiple isolated logical hosts.
- Each logical host has private CPU cores and local DRAM.
- All logical hosts map the same shared CXL memory region.
- Application-visible shared data lives in CXL-backed global memory.
- Each host may keep software-managed replicas of shared data in local DRAM.

Important caveat:

- This is a multi-host CXL pod emulation platform , not a true hardware multi-host non-coherent CXL pod.
- We must avoid relying on accidental hardware cache coherence inside the single machine.

Therefore the intended access discipline is:

- CXL shared memory is the authoritative global backing store.
- Host-local DRAM is a private replica/cache layer.
- All shared-memory accesses that matter semantically should go through the runtime, not arbitrary direct mutation.

3. LoomMem Core Mechanisms

3.1 Global Addressing

The design should not depend on host virtual addresses being identical.

The stable cross-host shared reference is:

- $GPtr = \langle region_id, offset \rangle$ or V1 simplification $GPtr = offset$

Each host resolves:

- $local_va = host_cxl_base + offset$

So the runtime contract is:

- applications and LoomPar use global pointers
- LoomMem resolves them to host-local mappings

3.2 CXL Memory Layout

The shared CXL region is logically divided into:

1. bootstrap/system metadata
2. global allocator metadata
3. global object/coherence metadata
4. CXL communication region
5. application shared data pool

This is the backbone for both LoomMem and LoomPar.

3.3 Global vs Local Metadata

This is one of the most important boundaries.

Global metadata in shared CXL memory:

- object/block offset and size
- allocation state
- generation
- version
- token owner
- global flags
- communication structures

Local metadata in each host's DRAM:

- replica local address
- cached / dirty state
- local version
- optional residency / hotness information
- local token-related transient state

Rule:

- global truth goes in CXL
- host-private cache state stays local

3.4 Coherence Model

The design explicitly avoids software MESI as a V1 target.

V1 coherence is:

- dynamic token ownership for writer serialization
- version-based freshness for readers
- release consistency for synchronization semantics

Separation of concerns:

- token answers: who may perform the next exclusive mutation

- version answers: whether a reader's local replica is fresh enough

3.5 Dynamic Token Ownership

Per shared block/object:

- there is exactly one token
- token ownership is exclusive
- token ownership is not data ownership

The token is a permission to execute the next mutable state transition.

The intended lifecycle is baton passing:

- allocator or first writer gets initial token
- current owner explicitly transfers token to the next owner
- avoid `NONE` → contender CAS race as the normal path

This gives the core invariant:

- at any time, exactly one host is the legal writer of a block

3.6 Version-Based Reader Freshness

Readers do not need the token.

Expected flow:

- on read, use local replica if present
- compare `local_version` and `global_version`
- if equal, read locally
- if stale, invalidate/reload from CXL and refresh local version

This is intentionally lightweight for read-mostly behavior.

3.7 Release Consistency Semantics

The document strongly prefers:

- release consistency for DRF programs

Instead of:

- per-write immediate global invalidation
- sequential consistency everywhere

The expected semantic shape is:

- ordinary reads/writes use local replicas
- release or barrier publishes writes
- acquire or barrier validates visibility/version

This aligns naturally with LoomPar barriers.

3.8 CXL-Resident Control Plane

The revised direction removes RDMA from the steady-state design.

CXL should carry:

- shared application data
- coherence metadata
- token-transfer coordination
- thread lifecycle control
- barrier control traffic

The preferred control primitive is:

- per-host-pair CXL-resident SPSC queues

Reason:

- fixed producer / fixed consumer
- avoids cross-host MPMC CAS/FAA complexity
- supports a single-writer discipline for mutable control fields

3.9 Ordering Requirements

Token transfer correctness depends on strict publication order:

1. finish dirty local data
2. write back / publish to CXL
3. execute visibility fence
4. publish new version / metadata
5. publish new token owner

6. send token grant

This is a hard implementation dependency and should be tested before full runtime work.

3.10 Granularity and Allocation

Coherence granularity should be configurable, not hard-coded.

Candidates include:

- 64B
- 256B
- 1KB
- 4KB
- 16KB

The design currently leans toward block/object-granular coherence, with 4KB as a practical V1 starting point.

Allocator design should be separated from coherence-token design:

- use global extent allocation
- give each host ownership of large extents
- perform host-local suballocation inside owned extents

This avoids a global lock on every `cxl_malloc()`.

4. LoomPar: What LeoPar Contributes

LeoPar provides the execution-layer model we want to adapt into LoomPar.

Its key idea is:

- DSM solves shared-data access
- the runtime should separately solve cross-node thread execution

LeoPar is intentionally:

- thread-centric
- Pthreads-like
- distributed
- pinned, not migratory

It does not move live stacks or continuations.

5. LoomPar Core Mechanisms

5.1 Programming Model

LeoPar exposes a distributed thread abstraction with:

- `leopar_init`
- `leo_thread_create`
- `leo_thread_join`
- `leo_barrier`
- `leo_world_size`
- `leo_rank`

Adapted into LoomPar, the conceptual API should remain similar:

- initialization/finalization
- distributed create/join
- barrier
- placement-aware execution

5.2 Pinned Cross-Host Threads

Each created thread:

- may execute locally or on a remote host
- gets a cluster-wide identity
- runs as a native local thread on the selected execution host
- stays pinned there until completion

This means LoomPar is not a task-migration framework.

The only thing that moves is the launch decision, not live execution state.

5.3 Home Rank and Global Thread Identity

LeoPar introduces:

- `GTID = <home_rank, local_tid>`

Semantically:

- `home_rank` : the creator-side lifecycle owner
- `execution_rank` : where the native thread actually runs

The home side keeps authoritative lifecycle metadata.

The execution side keeps only transient execution records.

This is a very good fit for LoomPar as well.

5.4 Remote Launch by Function Registration

LeoPar does not ship code or stack state.

Instead:

- functions are registered cluster-wide
- a remote create sends compact metadata:
 - GTID
 - `func_id`
 - argument payload

The remote side:

- resolves `func_id`
- reconstructs local invocation
- launches a native thread

This is the right execution model to preserve in LoomPar even if the transport changes from UCX/RDMA to CXL queues.

5.5 Lifecycle State Machine

The home-side lifecycle states are:

- `ALLOCATED`
- `LAUNCHING`
- `RUNNING`
- `COMPLETED`
- `JOINED`

Remote execution flow:

1. creator allocates GTID and home metadata
2. scheduler picks execution host
3. create request is sent
4. target host launches native thread
5. target sends create ack
6. thread runs and finishes
7. target sends complete notification
8. home host marks completed
9. join observes completion and reclaims metadata

This state machine should remain central in LoomPar.

5.6 Placement Scheduler

LeoPar schedules at create time.

It supports:

- explicit placement if caller specifies target rank
- round-robin as a cheap baseline
- locality + load aware scheduling

The locality-aware policy is based on:

- the dominant DSM region/address a thread is expected to use
- approximate per-rank load views
- a slack threshold that allows some extra load to preserve locality

This is especially valuable for LoomPar because LoomMem can expose richer locality signals than a generic DSM backend.

5.7 Barrier Semantics

LeoPar's barrier is:

- a distributed phase-boundary primitive
- separate from ordinary data-access semantics
- responsible for aligning thread progress
- dependent on the DSM backend to ensure data completion/visibility before barrier arrival

This matches LoomMem's release-consistency design almost exactly.

6. How LeoPar Must Change to Become LoomPar

The biggest architectural shift is:

- LeoPar originally used a DSM backend for data access and a UCX/RDMA control plane for execution control
- LoomPar should use LoomMem's CXL substrate for both data and control

So the adaptation path is:

- old LeoPar: DSM data plane + UCX control plane
- new LoomPar: LoomMem shared CXL memory + CXL queue control plane

Concretely, UCX control messages should become CXL-queue messages such as:

- CREATE_REQ
- CREATE_ACK
- COMPLETE_NOTIFY
- JOIN_WAKE or equivalent local completion path
- BARRIER_ARRIVE
- BARRIER_RELEASE
- token/coherence messages like TOKEN_REQ and TOKEN_GRANT

7. LoomMem <-> LoomPar Contract

This is the key interface we will need before coding.

LoomMem should provide LoomPar with:

- global pointer representation and translation
- shared allocation
- CXL-resident communication queues
- versioned read/write primitives or lower-level acquire/release hooks
- barrier-related completion/visibility hooks
- locality hints:
 - token owner
 - last writer
 - replica residency/hotness
 - dominant object/block placement

LoomPar should provide LoomMem with:

- thread creation context
- expected working-set hints for placement
- synchronization boundaries
- execution-host decisions that may affect data movement pressure

In other words:

- LoomMem knows where data state currently lives and how expensive it is to move coherence
- LoomPar decides where to run threads so execution follows favorable memory/coherence state

8. Most Important Joint Insight: Memory-Execution Co-Design

The strongest combined idea across the two documents is not just replacing transport.

It is:

- move computation toward coherence state

Instead of always moving token ownership and data state toward the thread, LoomPar can place the thread near:

- current token owner
- hottest replica
- last writer
- lowest expected coherence cost

This is the clearest research-level differentiator of CXLoom.

9. Practical V1 Boundaries

The two documents together suggest a realistic first implementation target:

- multi-NUMA logical-host emulation on one machine
- shared CXL region abstraction
- global offset-based addressing
- global allocator + per-host extents
- host-local replica cache

- dynamic token ownership per block
- version-based reader freshness
- release-consistency synchronization
- per-host-pair SPSC CXL queues
- distributed thread create/join/barrier on top of those queues
- initial placement policy using load + simple memory hint

What should not be overcommitted in V1:

- true hardware multi-host non-coherence claims
- full software MESI
- fully dynamic migrating execution
- hard-coded coherence granularity
- assuming specific CXL visibility primitives before measurement

10. Immediate Engineering Priorities

Before implementing the whole system, the documents imply this order:

1. CXL visibility litmus tests
2. shared-region bootstrap and global metadata layout
3. SPSC queue transport in shared CXL memory
4. global allocator and offset-based addressing
5. token/version coherence for one block granularity
6. LoomPar create/join control path over CXL queues
7. barrier + release/acquire integration
8. coherence-aware placement

This order is important because ordering and visibility semantics constrain everything above them.

11. Proposed Internal Naming

To keep the system clean in code, a good internal split is:

- loommem/
 - shared region bootstrap
 - allocator
 - gptra/addressing
 - metadata

- replica cache
- coherence/token/version
- cxl queue transport
- loompar/
 - API
 - thread table / GTID
 - dispatcher
 - function registry
 - scheduler
 - barrier
 - execution runtime
- common/
 - config
 - host/rank IDs
 - message formats
 - tracing
 - platform abstractions

12. Summary

CXLoom should be implemented as a two-layer co-designed runtime:

- LoomMem is the software-coherent shared-memory substrate over CXL, with global metadata in CXL, private replicas in host DRAM, dynamic-token write serialization, version-based reader freshness, release consistency, and CXL-resident SPSC queues.
- LoomPar is the distributed thread runtime above LoomMem, preserving Pthreads-like create/join/barrier semantics, using home-owned global thread lifecycle management, metadata-only remote launch, pinned execution, and placement guided by load plus memory/coherence locality.

The most important architectural decision is that CXLoom should not treat memory and execution as separate afterthoughts. LoomMem exposes coherence state, and LoomPar should use that state to place threads where the total execution plus coherence cost is lowest.